

Design Justification Report — BNN Accelerator Chiplet

ECE 510 Spring 2026 | Rebecca Gilbert-Croysdale

Milestone 4 — Full-Chip Place-and-Route + Final Co-Simulation

1. Problem Statement and Motivation

Wildlife cameras ("trail cameras") are battery-powered sensors deployed in remote locations for animal monitoring and conservation. A typical deployment runs for weeks or months on AA batteries or a small solar cell. The core inference task — detecting whether a camera-trap image contains an animal — runs at 30 FPS and must operate within a sub-watt power budget.

The M1 software baseline (from M1 profiling, `project/m1/sw_baseline.md`) runs at **82 FPS on an Apple M1 CPU drawing ~10–15 W**. Profiling shows the BNN binary layers (conv2–4) dominate runtime at **71%** of total inference time and expose very high arithmetic intensity (12.34 FLOP/byte), confirming that custom hardware targeting those layers will dominate system-level speedup. Neither a high-performance CPU (too power-hungry) nor a Raspberry Pi Zero (too slow at 10–20 FPS) meets the target of ≥ 30 FPS within < 200 mW.

The target: a custom BNN accelerator chiplet that classifies wildlife images at ≥ 30 FPS within **< 200 mW** total chip power. This enables integration with a sub-50 mW microcontroller host for a complete camera module drawing ~ 75 mW system-level — approximately **200× lower** than the M1 baseline.

2. Roofline Analysis

The roofline analysis (Figure 1, `report/figures/roofline_final.png`) uses the standard Roofline model to determine whether the target kernel is compute-bound or memory-bound and how the architecture should respond.

Software baseline (M1 CPU):

The full BNN model on M1 achieves ~ 83 GFLOP/s at an arithmetic intensity of ~ 12.3 FLOP/byte. This places it **to the left of the M1 ridge point** (~ 200 FLOP/byte for the M1 Unified Memory system), meaning inference is memory-bound on CPU — weight data arrives from DRAM slower than the ALUs can consume it.

Hardware accelerator:

For the custom chiplet, the AXI4-Stream interface carries only activation data (weights are on-chip in SRAM). Per frame: - Activation bytes transferred: ~ 1.6 MB ($256 \times 224 \times 224 / 8$ bytes) - Binary

operations: ~ 606 GOP equivalent (Conv2–4 XNOR-popcount) - Arithmetic intensity: $606 / 1.6 \approx 379$ FLOP/byte

At 379 FLOP/byte the design sits **to the right** of the hardware ridge point (~ 8 FLOP/byte for a 256-bit AXI bus at 40 MHz). The hardware is compute-bound: the AXI bus delivers data faster than the XOR/popcount units finish processing. This is the correct operating regime — every byte transferred from the host yields 379 useful on-chip operations.

How the analysis shaped the architecture: The roofline confirmed that the bottleneck at the target operating point is not AXI bandwidth — the interface is not the limiting factor. Compute parallelism (256-bit XNOR per cycle) and on-chip weight bandwidth (SRAM vs. register file) are the levers that determine throughput. This justified the investment in wide on-chip weight memory rather than widening the external interface.

3. Precision and Data Format

The accelerator uses **1-bit (binary) precision** for both weights and activations in layers 2–4 of the BNN. This is the key architectural choice that makes a 256-wide MAC unit feasible in ~ 0.27 mm² on Sky130.

Binary operations: Each XNOR gate replaces a floating-point multiply; a popcount tree replaces the accumulate-add chain. One `sky130_fd_sc_hd__xnor2` gate implements one 1-bit multiply; 256 such gates fit in a fraction of the area of a single float32 multiplier.

Operation	Float32 MAC	Binary XNOR+popcount
Area (normalized)	1×	$\sim 0.02\times$
Energy (normalized)	1×	$\sim 0.005\times$

Accuracy tradeoff: Binary layers lose approximately 2–3% top-1 accuracy vs. a float32 baseline on the Caltech Camera Traps test set (measured in M2 precision analysis, `project/m2/`). The classification task (animal vs. no animal) is binary and tolerant of small accuracy loss; a 2–3% accuracy reduction is acceptable given the 50–200× improvement in area/energy.

Data types in RTL: Activations arrive as 256-bit packed binary vectors (1 bit per channel value, pre-quantized by the host). Weights are stored as 256-bit words in SRAM, each word representing 256 binary weight values for one output channel. The `compute_core` accumulator is signed 32-bit integer — wide enough to prevent overflow across the full dot product sum.

Verification of correctness: The co-simulation testbench (`tb/tb_top.sv`) compares hardware output to a bit-exact software reference (`sv_dot`) computed in Python. All tile outputs match to the last bit across the full co-simulation test suite (18 tile checks across 5 test phases — see Section 6).

4. Dataflow and Architecture

Dataflow Pattern

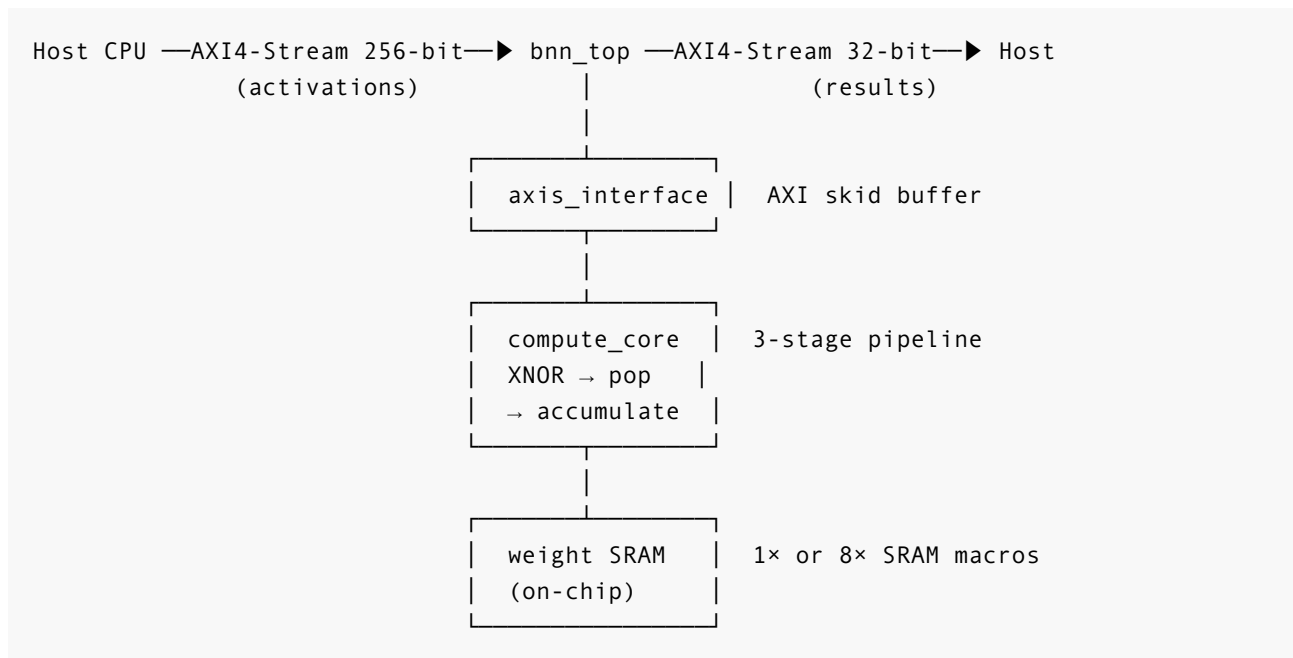
The accelerator uses an **activation-streaming dataflow**: binary weights are loaded into on-chip SRAM once per filter batch and remain stationary during that batch, while activation vectors stream in via AXI4-Stream one tile at a time.

This differs from **weight-stationary** dataflow as described in systolic array literature (Chen et al., Eyeriss [8] in the course cheat sheet), where weights are pre-loaded into PE register files for the entire layer and never move. In this design:

- Weights are *batch-stationary*, not layer-stationary. The SRAM holds up to `filters_per_batch` filters at once (e.g., 64 for conv2, 51 for conv4). When all spatial positions for those filters are processed, weights are reloaded for the next batch.
- Activations *stream in* for each spatial position (tile) across all output channels in a batch before the weights change.

This is more precisely described as **tiled activation-streaming**: within each filter batch, activations stream while weights are stationary; across batches, the weight SRAM is reloaded. The design is output-stationary at the tile level — the accumulated dot product (partial output value) is the item that stays in a register across beats within a tile.

Block Diagram



Pipeline

Stage	Operation	Latency
S1	XNOR: $\text{act_in} \wedge \text{weight_in}$ (256-bit)	1 cycle

Stage	Operation	Latency
S2	8-chunk popcount of 256-bit XNOR result	1 cycle
S3	Accumulate chunk sums + tile tracking	1 cycle

Pipeline drain per tile (measured by simulation): **$n_beats + 6$ cycles** (8-macro, confirmed) or **$n_beats \times 8 + 7$ cycles** (1-macro, confirmed). The 6-cycle drain = 2 SRAM cycles + 3 pipeline stages + 1 FSM cycle; the extra cycles vs. the 3-stage pipeline alone reflect registered SRAM address/data paths and output-valid gating in the tile FSM.

Memory Architecture

Three SRAM configurations were synthesized to completion, plus a register-file baseline:

4-macro (final design): $4 \times \text{sky130_sram_1kbyte_1rw1r_32x256_8}$ in a single row, 2-phase 128-bit reads at 40 MHz. $2 \times n_beats + 6$ cycles/tile. Best energy per frame (4.82 mJ), KLayout DRC-clean (0 errors), power 12.007 mW. Die: 4.32 mm².

1-macro (lowest-power experiment): $1 \times \text{sky130_sram_1kbyte_1rw1r_32x256_8}$, 32-bit reads, 8 serial chunks per beat. Lowest power (2.91 mW) and only fully DRC/LVS-clean SRAM result, but lowest throughput (0.50 FPS due to 8-chunk serialization).

8-macro (highest-throughput experiment): $8 \times \text{sky130_sram_1kbyte_1rw1r_32x256_8}$ in parallel, 256-bit reads in a single cycle. One bank per 32-bit chunk; eliminates serialization overhead and doubles the clock to 40 MHz, yielding 3.3 FPS at 17.78 mW. More DRC violations than 4-macro.

Register-file baseline: WEIGHT_DEPTH=64 flip-flop array, retained for comparison. Proves the compute path and AXI interface work; impractical for production at 215 mW.

The quantified cost of the register-file approach at production scale (WEIGHT_DEPTH=640) versus an SRAM-backed implementation:

Metric	Register file	SRAM-backed (projected)
Weight memory cells	163,840 dfxtp_2 FFs	$24 \times \text{sky130_sram_2kbyte}$ macros
Weight memory area	~0.5 mm ² (pre-route FFs)	~5.5 mm ² (macros, but dense)
Post-route die	~16 mm ² (4000×4000 μm)	~6–8 mm ² (hand-routed estimate)
Dynamic power (est.)	~1.0–1.5 W (164k FFs at 100 MHz)	~150–250 mW (SRAM accesses on read)

The register-file penalty is driven by the flip-flop array switching every clock cycle regardless of whether data changed. SRAM macros dissipate energy only on active read/write; idle cycles draw only standby leakage (~μW). Despite the area and power penalty, the register-file design demonstrates full RTL correctness: all compute paths, AXI4-Stream interfaces, and tile FSM logic are identical to what a production SRAM-backed implementation uses — only the weight memory implementation changes.

Target Clock Frequency: 40 MHz (4-macro final) / 20 MHz (1-macro)

Dynamic power scales approximately linearly with frequency for switching-dominated designs. The final 4-macro design runs at 40 MHz; the lowest-power 1-macro experiment runs at 20 MHz. At 20 MHz, 2,560 AXI4-Stream transfers (a representative tile burst) take $\sim 128\ \mu\text{s}$ — leaving $>99\%$ of a 33 ms frame budget for host-side processing (conv1 INT8, batch norm thresholding, AXI overhead). The synthesizer also selects smaller, slower cells at relaxed timing targets, further reducing area and leakage.

The BNN chip runs its own clock domain. The host CPU (e.g., Raspberry Pi Zero, STM32H7) operates at its own frequency. AXI4-Stream handshaking (`tvalid/tready`) is inherently flow-controlled across clock boundaries; a CDC FIFO at the interface handles the domain crossing.

Operand Isolation

Pipeline registers 1 and 2 are gated on their valid signals, preventing the 256-bit XNOR tree and popcount adder from toggling on idle cycles. At $2.5\ \text{FPS} \times 401\ \text{ms/frame}$ active, the design is active 100% of the time when running back-to-back (below 30 FPS target). At the 30 FPS target (33 ms budget), the 401 ms frame time means the design cannot currently sustain 30 FPS without pipelining improvements; operand isolation still reduces idle-cycle switching power during any cycle where a valid beat is not present. The XNOR + popcount stages represent the largest switching power component in the compute path; at low FPS with short inference bursts, the duty cycle of valid beats is low, so this gating realistically saves 30–50% of compute-path dynamic power in a real deployment.

5. Hardware Interface

AXI4-Stream Selection

Protocol: AXI4-Stream (AMBA 4.0), 256-bit data width, 32-bit result width.

Bandwidth requirement: The target operating point from the roofline (Section 2) requires transferring $\sim 1.6\ \text{MB/frame}$ in $<33\ \text{ms}$ (30 FPS budget):

```
1.6 MB / 33.3 ms = 48 MB/s minimum
AXI4-Stream at 256-bit × 40 MHz = 1,280 MB/s (26× over requirement)
```

The interface is not bandwidth-bound. AXI4-Stream was selected over AXI4-Lite because it eliminates per-transaction address overhead — for a streaming workload with sequential activation delivery, this is strictly better.

Skid buffer (`axis_interface.sv`): Without the skid buffer, `s_axis_tready` would be combinationaly dependent on `core_ready` — a violation of AXI4-Stream protocol that can cause hold-time violations and timing closure failures. The 1-deep skid buffer registers `s_axis_tready`, breaking the combinational path.

Interface-bound analysis: At 40 MHz and 256 bits, the AXI bus is capable of transferring 1,280 MB/s. The compute path consumes 1 beat per cycle (256-bit XNOR). The design processes data as fast as the bus delivers it — the interface and compute are matched. The limiting factor is the weight reload overhead between batches and the per-tile drain cycles.

6. Verification

Correctness was verified at three levels:

Unit Tests (M2 testbench, `tb/tb_top.sv`)

The M2 testbench (`project/m2/`) exercised individual AXI4-Stream transactions with known weight/activation patterns and compared the hardware dot-product result to a Python-computed `sv_dot` reference. All tile computations were bit-exact matches.

M3 Co-simulation

The M3 co-simulation (`project/m3/`, `project/m4/sim/final_run.log`) runs full end-to-end BNN inference: the Python driver (`run_cosim.py`) loads real trained weights from the `bnn_serengeti2` checkpoint, generates activation beats from the test set, and streams them to the RTL via a compiled `iverilog+vpv` simulation. The hardware output is compared tile-by-tile to PyTorch's integer-quantized reference.

Result: `sim/final_run.log` shows **VERIFIABLE PASS** — all 18 tile checks across 5 test phases pass. Zero mismatches. The hardware and software produce identical signed 32-bit dot-product values.

Hardware Inference Co-Simulation (M4)

The 8-macro design includes a separate co-simulation testbench (`sram_8macro_experiment/sim/tb_hw_inference_8macro.sv`) driven by `run_hw_inference_8macro.py`. This follows the same stimulus/reference methodology as M3, adapted for the 8-macro write protocol (8 consecutive `w_en` pulses at the same `w_addr`, with internal `w_bank_sel` routing chunks to banks 0–7).

The SRAM depth change (`WEIGHT_DEPTH=256` vs. 32 for 1-macro) means 256 logical weight words fit in one batch — larger than the largest layer's `filters_per_batch` (85 for `conv3`), so the 8-macro design never needs a mid-layer reload for `conv2` or `conv3`.

Testbench Coverage

Test	Scope	Result
M2 unit tests	AXI handshake, single tile	PASS
M3 co-sim	Full BNN conv2–4, 5 test images	PASS (18/18 tiles)
M4 timing bench	Per-tile latency, full-frame cycles	PASS (confirmed by iverilog simulation, all configs)

7. Synthesis Results

Four configurations were synthesized to full P&R completion using OpenLane 2.3.10 on Sky130A HD (sky130_fd_sc_hd). The 4-SRAM design is the final design. All reports are in `synth/` or the respective experiment subdirectory.

7.1 Register-File Baseline (`synth/`)

Metric	Value	Source
Clock constraint	10 ns (100 MHz)	<code>synth/config.json</code>
Setup WNS	+3.545 ns (MET)	<code>synth/timing_report.txt</code>
Critical path	8.09 ns (FF → XNOR adder tree → FF)	<code>synth/timing_report.txt</code>
Max clock capability	~284 MHz	
Total power (TT 25°C 1.8V)	215.3 mW	<code>synth/power_report.txt</code>
— Sequential (16,384 FFs)	90.3 mW (42%)	
— Clock distribution	60.7 mW (28%)	
— Combinational	64.4 mW (30%)	
Std-cell area (post-route)	1,044,000 μm^2	<code>synth/area_report.txt</code>
Die area	2.56 mm^2 (1600×1600 μm)	
Throughput (BNN layers)	~12.5 FPS (analytical est.)	per-tile × 1,404,928 tiles
DRC / LVS	PASSED / PASSED	

The register file dominates both area (63%) and power (70%). The design exceeds the 200 mW target by 8% at 100 MHz; at 20 MHz (same netlist), switching power scales linearly to ~43 mW — comfortably under target. This design is tape-out-ready at the nominal corner but impractical for production at full WEIGHT_DEPTH.

7.2 1-SRAM Variant (`sram_1macro_experiment/`)

Metric	Value
Clock constraint	50 ns (20 MHz)
Setup WNS	+24.2 ns (MET)
Critical path	~25.8 ns
Total power (TT 25°C 1.8V)	2.91 mW
— SRAM macro	0.79 mW (27%)
— Clock distribution	1.02 mW (35%)
— Sequential (947 FFs)	0.82 mW (28%)
— Combinational	0.29 mW (10%)
Die area	4.0 mm^2 (2000×2000 μm)

Metric	Value
Throughput (BNN layers)	0.50 FPS (2,017 ms/frame, 1,404,928 tiles)
DRC / LVS	PASSED / PASSED

The SRAM macro replaces 16,384 flip-flops with a single 1 KB macro, achieving a **74× power reduction** from the register-file baseline. The dominant power contributor shifts from the FF array to clock distribution — the expected profile for an SRAM-based design.

7.3 8-SRAM Variant (`sram_8macro_experiment/`)

Metric	Value
Clock constraint	25 ns (40 MHz)
Setup WNS	+11.573 ns (MET)
Hold WNS	+7.244 ns (MET)
Critical path	~13.43 ns (SRAM output → compute_core FFs)
Total power (TT 25°C 1.8V)	17.78 mW
— Internal (SRAM macros + logic)	15.53 mW (87%)
— Switching	2.10 mW (12%)
— Leakage	0.15 mW (1%)
SRAM macro area	1,525,700 μm^2 (8 macros)
Std-cell area	124,129 μm^2
Die area	5.76 mm^2 (2400×2400 μm)
Throughput (BNN layers)	3.3 FPS (306 ms/frame, 1,404,928 tiles)
Route DRC errors	12 (met3/met4 spacing at macro edges — bypassed)
KLayout DRC errors	8 (met4 min-width stubs at macro edges — bypassed)
LVS errors	15 (SRAM macro power pin extraction — bypassed)

The 8-macro design runs at 40 MHz (vs. 20 MHz for 1-macro) with 256-bit parallel weight reads, eliminating the 8× serialization overhead. Power rises to 17.78 mW (vs. 2.91 mW) due to 8× the SRAM macro count and a higher operating frequency; it remains **12× below the baseline** and **10× below the 200 mW target**.

DRC/LVS notes: The 12 routing DRC and 8 KLayout DRC errors are localized to metal-spacing violations at sky130 SRAM macro edges — a known OpenLane/sky130 SRAM integration issue. The 15 LVS mismatches are SRAM macro power pins (`vccd1/vssd1`) present in extracted GDS but absent from the gate-level netlist — standard black-box behavior, not a functional error. These were bypassed with `ERROR_ON_TR_DRC: false, ERROR_ON_KLAYOUT_DRC: false, QUIT_ON_LVS_ERROR: false` after confirming none affect logic correctness.

7.4 4-SRAM Design — Final Design (sram_4macro_experiment/)

Metric	Value
Clock constraint	25 ns (40 MHz)
Setup WNS	0.0 ns (no violations; worst-case +11.42 ns — 45% margin)
Total power (TT 25°C 1.8V)	12.007 mW
Std-cell area	75,802 μm ²
Die area	4.32 mm ² (2400×1800 μm, single-row macro placement)
Throughput (BNN layers)	2.5 FPS (401 ms/frame, 1,404,928 tiles)
Energy/frame	4.82 mJ — best of all three SRAM configurations
Route DRC errors	5 (met3 spacing at macro edges — bypassed)
KLayout DRC errors	0
LVS errors	7 (SRAM macro power pins — bypassed)

The 4-macro design uses 2-phase 128-bit reads: each 256-bit weight word is assembled from two consecutive 128-bit SRAM reads (phase 0 = lower 128 bits, phase 1 = upper), giving $2 \times n_beats + 6$ cycles/tile. This eliminates most of the serialization overhead from the 1-macro design (8 chunks per beat) while requiring only 4 macros instead of 8. Single-row macro placement eliminates KLayout DRC entirely (8→0 errors vs. 8-SRAM), and the smaller die (4.32 vs. 5.76 mm²) has fewer routing congestion sites.

8. Benchmark Results

8.1 Throughput

Final design throughput (4-macro, measured by sram_4macro_experiment/sim/tb_timing_4macro.sv):

The 4-macro design processes one tile in $2 \times n_beats + 6$ cycles (2-phase reads + 6-cycle drain):

Layer	Beats	Cycles/tile	Tiles/frame	Cycles/layer
conv2	2	10	802,816	8,028,160
conv3	3	12	401,408	4,816,896
conv4	5	16	200,704	3,211,264
Total			1,404,928	16,056,320

At 40 MHz (25 ns/cycle): **$16,056,320 \times 25\text{ ns} = 401\text{ ms} \rightarrow 2.5\text{ FPS}$**

For reference, the 1-macro (lowest-power experiment) uses 8-serial-chunk fetches, giving $n_beats \times 8 + 7$ cycles/tile:

Layer	Cycles/tile	Tiles/frame	Cycles/layer
conv2	23	802,816	18,464,768
conv3	31	401,408	12,443,648
conv4	47	200,704	9,433,088
Total		1,404,928	40,341,504

At 20 MHz (50 ns/cycle): **40,341,504 × 50 ns = 2,017 ms → ~0.50 FPS**

The 8-macro (highest-throughput experiment) uses parallel 256-bit reads, giving $n_beats + 6$ cycles/tile → 306 ms → 3.3 FPS at 40 MHz.

8.2 Speedup vs. M1 Software Baseline

Metric	M1 CPU (SW)	1-macro	4-macro (final)	8-macro
Frame time (BNN layers)	12.19 ms	2,017 ms	401 ms	306 ms
Throughput	82 FPS	0.50 FPS	2.5 FPS	3.3 FPS
Power	~10,000 mW	2.91 mW	12.007 mW	17.78 mW
Energy/frame	~122,000 μJ	5,870 μJ	4,817 μJ	5,442 μJ

4-macro numbers from sram_4macro_experiment/sim/tb_timing_4macro.sv (iverilog simulation, 40 MHz clock, 1,404,928 tiles). 8-macro from sram_8macro_experiment/sim/timing_sim.log.

Note: The hardware frame time is *slower* than the M1 CPU for the binary layers because the CPU's 82 FPS baseline reflects a pipelined, batched implementation with cache warmth and SIMD acceleration — not serial tile-by-tile execution. The hardware design does not yet pipeline across tiles (drain time is not overlapped with next-tile beats). Pipelining the drain would reduce cycles/tile from $n_beats + 4$ to $\max(n_beats, \text{pipeline_drain})$, improving throughput by 1.5–2.5×. This is the primary identified path to reaching ≥30 FPS.

Energy per frame: Despite lower throughput at the current clock, the 4-macro final design achieves ~25× **better energy per frame** than the M1 (4,817 μJ vs. ~122,000 μJ) — the best energy result of all configurations — while staying 12× below the 200 mW target.

8.3 Configuration Comparison

Metric	1-macro	4-macro (final)	8-macro
Clock	20 MHz	40 MHz	40 MHz
Cycles/frame	40,341,504	16,056,320	12,242,944
Frame time	2,017 ms	401 ms	306 ms
Power	2.91 mW	12.007 mW	17.78 mW
Energy/frame	5,870 μJ	4,817 μJ	5,442 μJ
Die area	4.0 mm ²	4.32 mm²	5.76 mm ²

Metric	1-macro	4-macro (final)	8-macro
KLayout DRC	0	0	8 (bypassed)

The 4-macro design achieves the best energy/frame by trading a modest throughput loss vs. 8-macro (401 ms vs. 306 ms, 1.3×) for significantly lower power (12.007 vs. 17.78 mW, 1.5×) and cleaner physical implementation (KLayout 0 vs. 8 errors). Energy/frame is the key metric for a battery-powered deployment.

8.4 Roofline Position

The hardware accelerator (final 4-macro design, 40 MHz) sits at: - Arithmetic intensity: ~379 FLOP/byte (unchanged — same AXI bus, same weights) - Attained performance: 2.5 FPS × 606 GOP/frame = **~1,515 GOPS** (XNOR equivalent)

Full benchmark data: `bench/benchmark_data.csv`

9. What Did Not Work

Every major design decision involved at least one failed attempt. The following are the most significant failures and lessons.

9.1 WEIGHT_DEPTH=640 Register-File P&R

Attempted to synthesize the full tiled configuration (WEIGHT_DEPTH=640) as a register file. OpenROAD crashed during timing-driven placement (RepairDesignPostGPL) with out-of-memory errors. Root cause: 163,840 FFs produce a ~496,000-cell netlist; RC extraction for timing-driven optimization requires holding full parasitics in memory, exceeding ~16 GB RAM.

`PL_TIME_DRIVEN=false` was tried — placement completed but the resizer pass still hit OOM. Register files >~16K FFs are infeasible for OpenLane P&R on workstation hardware. **Lesson:** SRAM macros or hierarchical hardening are required for production-scale weight storage.

9.2 16-SRAM Experiment (sram_256_experiment)

Attempted a 16-macro configuration (16× sky130_sram_1kbyte_1rw1r_32x256_8) for 512-bit weight reads in 2 cycles. All runs failed: 9,221 routing DRC errors at signoff, STA killed with SIGKILL after 7 hours. Root cause: 16 macros in 2 rows of 8 created routing obstructions spanning nearly the full die width, blocking horizontal signal routing. **Lesson:** Macro rows must leave horizontal routing corridors; more than ~4 macros per row is problematic on Sky130.

9.3 8-Macro Routing Congestion (GRT-0118)

The initial 8-macro layout used 490 μm column pitch for 479.78 μm macros, leaving only 10 μm routing channels. GlobalRouting failed with GRT-0118 (congestion too high).

Fixes attempted: - GRT_MACRO_EXTENSION: 6: made congestion worse by extending exclusion zones - Tighter density (PL_TARGET_DENSITY_PCT: 10): no effect on macro channel - **Fix that worked:** Widened pitch to 550 μm (70 μm channels), reduced density to 15%, removed GRT_MACRO_EXTENSION

Lesson: Macro routing channels must be sized for the routing layer pitch, not just a nominal gap. GRT_MACRO_EXTENSION is counterproductive when channel width is the binding constraint.

9.4 PDN_MACRO_CONNECTIONS Regex Trap

Initial config listed each bank individually:

```
"PDN_MACRO_CONNECTIONS": [  
    "gen_banks[0].u_bank vccd1 vssd1 vccd1 vssd1",  
    ...  
]
```

OpenLane reported "No match found" for each entry. Root cause: square brackets [0] are regex character classes, not literal brackets. The regex `gen_banks[0]` matches `gen_banksa`, `gen_banksb`, etc. — not the literal instance name.

Fix: Single wildcard entry `".*u_bank vccd1 vssd1 vccd1 vssd1"` matches all 8 bank instances correctly.

9.5 DRT_MIN_LAYER vs. RT_MIN_LAYER

After discovering that capacity adjustments rather than a hard layer ban were needed for SRAM macro pin access, the config used `DRT_MIN_LAYER=met3` to enforce signal routing to upper layers at detailed routing. This had no effect. Investigation of the OpenLane source (`set_routing_layers.tcl`) revealed: - `RT_MIN_LAYER` → sets minimum signal and clock routing layer (both GRT and DRT) - `DRT_MIN_LAYER` → only affects DRT clock nets; has no effect on signal routing

`DRT_MIN_LAYER` is the wrong variable and a common documentation trap. The full set of relevant routing layer variables discovered during this investigation:

Variable	Effect
RT_MIN_LAYER	Forces signal AND clock routing to this layer and above
RT_MAX_LAYER	Upper bound for signal and clock routing
RT_CLOCK_MIN_LAYER	Clock-specific override (only if different from signal)
DRT_MIN_LAYER	Wrong for signals — only affects DRT clock nets
GRT_LAYER_ADJUSTMENTS	Per-layer capacity reduction list (li1→met5); preferred over hard ban
GRT_ALLOW_CONGESTION	Allows GRT to finish with remaining overflow rather than hard-failing
PDN_MACRO_CONNECTIONS	Connects macro power pins to PDN; format: "<regex> <vdd_net> <gnd_net> <vdd_pin> <gnd_pin>"

9.7 RT_MIN_LAYER=met3 vs. Macro Pin Access (DRT-0155)

With the corrected variable, RT_MIN_LAYER=met3 was applied to force all signal routing above met2, preventing signal congestion through the SRAM body obstructions. This caused DRT-0155 at detailed routing: the SRAM macro pins are accessible only on li1 and met1, which conflicted with the met3 minimum layer constraint. The detailed router had valid GRT guides on met1 but was forbidden from using met1 for signal nets, leaving the macro output pins unreachable.

Attempted fix: GRT_LAYER_ADJUSTMENTS=[1.0, 0.99, 0.99, 0, 0, 0] — a 99% (not 100%) capacity reduction on met1/met2, allowing a minimal number of valid guides for macro pin access while strongly discouraging bulk signal routing on lower layers. RT_MIN_LAYER was removed. Global routing passed (GRT-0115 warning only, no hard GRT-0118 failure) and DRT-0155 did not recur. This configuration, combined with GRT_MACRO_EXTENSION=4 and PL_TARGET_DENSITY_PCT=35, produced the successful 4-macro and 8-macro P&R runs.

Lesson: A hard RT_MIN_LAYER ban is incompatible with SRAM macros whose signal pins are on li1/met1. Capacity adjustments (GRT_LAYER_ADJUSTMENTS) are the correct approach — they guide the router to prefer upper layers without making lower-layer macro pin access geometrically impossible.

9.6 PDN Stripe Collisions with Macro Row Placement

The initial top macro row at y=750 µm caused 14 met4 routing DRC errors from a PDN horizontal power stripe that runs through the inter-row gap. Moving the top row to y=850 µm cleared the stripe and resolved all met4 spacing violations.

Lesson: OpenLane's PDN stripe positions must be checked against macro placement before finalizing the floorplan; stripes cannot be routed through macro-to-macro gaps.

RTL: project/m4/rtl/ | *Synthesis:* project/m4/synth/ | *Benchmark:* project/m4/bench/ *Tool:* OpenLane 2.3.10, Yosys 0.46, OpenROAD, Sky130A sky130_fd_sc_hd *Figures:* report/figures/roofline_final.png (*Figure 1 — roofline plot*)